

Project 2: Providing recommendations

In this project we aim to compute predictions and, in case of a negative outcome prediction, providing recommendations on the best activities to perform to achieve a positive outcome by leveraging the transparency of decision trees.

Use the *boolean* encoding and write the following Python functions:

- *extract_recommendations* (*tree*, *feature_names*, *class_values*, *prefix_set*): a Python function that takes as input a decision tree *tree*, the names of the features *feature_names*, a list of class values *class_values* (in our case ["true","false"]), and a set of prefix traces *prefix_set* (e.g., the prefix traces in the test set). The function should extract, for each prefix trace with a negative (false) prediction, the conditions (not already verified in the prefix trace) along the path of the tree compliant to the prefix that leads to a positive outcome (label true) with the highest confidence (probability of the class). In other terms the function should return, for each prefix trace of the *prefix_set*, a set of conditions that is
 - empty for prefix traces with a positive prediction and for negative predictions for which no recommendation can be extracted from the decision tree
 - the set of conditions along the path in the tree compatible with the conditions already set in the tree and with the highest confidence score (class probability) for prefix traces with a negative prediction, for which recommendations can be extracted from the decision tree.

Please note that the problem can be decomposed in simpler steps:

- extraction of the positive paths
- for each prefix trace:
 - filtering of the positive paths compliant with the prefix itself
 - filtering of the path with the highest confidence score
 - extraction of the conditions that do not already occur in the prefix trace
- *evaluate_recommendations* (*test_set*, *recommendations*): a Python function that takes as input a set of traces *test_set* and the recommendations, containing for each prefix trace of the test set the recommendation, and returns an evaluation of the recommendations.

To this aim you can consider the following classification

- if the recommendation is followed in the complete trace and the ground truth value of the outcome is positive, then the recommendation is classified as a **true positive**
- if the recommendation is not followed in the complete trace and the ground truth value of the outcome is negative, then the recommendation is classified as a **true negative**

- if the recommendation is followed in the complete trace and the ground truth value of the outcome is negative, then the recommendation is classified as a **false positive**
- if the recommendation is not followed in the complete trace and the ground truth value of the outcome is positive, then the recommendation is classified as a **false negative**

Starting from this classification, the function could return a sort of approximated F-measure score for recommendations (*F-measure_recc*) or another metrics that you define and explain.

The Production log, which is a real-life log containing data from a manufacturing process, is labelled based on whether the traces were fast (cycle time < average cycle time) or slow (cycle time >= average cycle time): in the former case the label is set to *true*, in the latter to *false*.

- Compute the *prefix_training_log* containing the prefixes of *prefix_length* of the training set (80%) of the labelled log Production (Production_avg_mean_training_0-80.xes.zip)
- Train a decision tree model (and optimize its hyperparameters) with the *prefix_training_log* by using the *boolean* encoding.
- Compute the *prefix_testing_log* containing the prefixes of *prefix_length* of the testing set (20%) of the labelled Production log (Production_avg_mean_testing_80-100.xes.zip).
- Compute the accuracy of the predictions (using classical prediction metrics, such as F-measure or accuracy) on the *prefix_testing_log*. Use padding in case traces are shorter than the chosen prefix length.
- Compute the recommendations for the prefixes in the *prefix_testing_log*, whenever it is possible, i.e., whenever a recommendation is returned.
- Qualitatively inspect and explain the returned recommendations.
- Compute the *F-measure_recc* or another metrics you define to evaluate the accuracy of the recommendations.
- Compare and discuss the prediction and the recommendation accuracy results for different prefix lengths (e.g., 5 and 10).

We expect to receive:

- the commented Python code of the required functions;
- a main function calling the training and the the testing functions, as well as the function computing the recommendations and the one evaluating them;
- the instructions to run the code;
- a report describing the overall structure of the code and reporting and discussing the results obtained for predictions and recommendations.